Transcription SaaS

A Whisper-Based Speech-to-Text Microservice Platform

5CCSACCA – Cloud Computing for Artificial Intelligence

King's College London

GitHub repository: `https://github.com/5CCSACCA/resit-coursework-ra1425`

August 2, 2026

1 System Overview and Design Decisions

This project implements a transcription SaaS built around OpenAI's Whisper speech recognition model (Radford et al., 2022). The system exposes a RESTful API, built with FastAPI (Ramírez, 2024), through which registered users submit audio files and receive text transcriptions.

The input is an audio file (WAV, MP3, WebM, OGG or M4A, up to 25MB) submitted with a bearer token identifying the user. The output is a JSON object containing the transcribed text, the detected language, and the processing duration.

Because transcription is not instantaneous, the system uses an asynchronous job pattern: a request creates a job record immediately, transcription runs in a background service, and the client polls for completion, rather than blocking the connection for the duration of inference.

A PostgreSQL database underpins the system (PostgreSQL Global Development Group, 2024), chosen for its support for foreign-key relationships and transactional consistency. Three normalised tables, `users`, `jobs` and `transcripts`, separate account data, job state, and results, since a job may exist, and fail, before any transcript is produced.

2 Architecture

The system runs as five containers under Docker Compose: two application services, and Postgres, Prometheus and Grafana supporting them, shown in Figure 1.

The api-service handles accounts, login and incoming requests, and does not run any part of the Whisper model, keeping its container small. The transcription-service does the audio work, running the general Whisper model through faster-whisper, and a second, fine-tuned model for Scottish accented speech.

The two services communicate over the network Docker Compose builds automatically; the api-service reaches the transcription-service by its service name rather than an IP address (Docker Inc., 2024).

The transcription-service cannot be reached from outside the Docker network: the compose file uses `expose` instead of `ports` for it, so only other containers can reach it. The api-service is the only service published to the host, on port 8000, so every request passes through login and rate limiting before it can reach the model.

Both transcription routes, general and Scottish, work the same way: the api-service creates a job record, validates the upload, forwards it to the transcription-service, and saves the result. This logic sits in one function both routes call, so a fix in one route cannot be missed in the other.

The transcription-service is limited to 4 CPUs and 16GB of memory directly in the compose file, tested by sending several requests at once. All requests finished and CPU use stayed under the limit, confirming it works within the coursework's hardware limit.

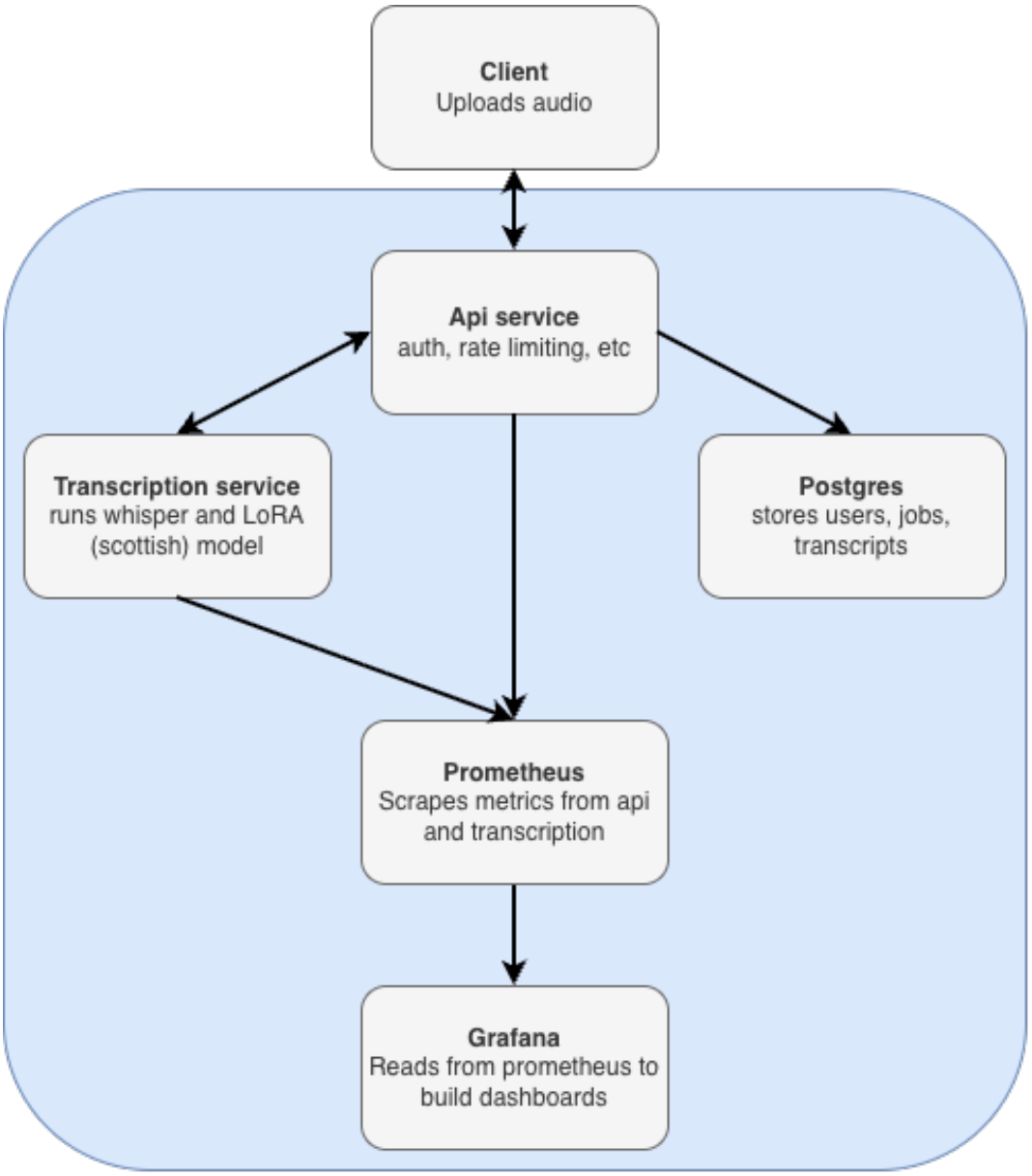

Figure 1: Microservice architecture. The client only ever reaches the api-service, which forwards transcription requests to the internal transcription-service. Prometheus scrapes both services and Grafana reads from Prometheus.

3 The Model

The system uses Whisper for speech to text (Radford et al., 2022). The general endpoint runs the base model through faster-whisper, a CTranslate2 reimplementation of Whisper (OpenNMT, 2024), chosen for CPU speed under the coursework's 4 vCPU limit. The model uses int8 quantization, halving memory use and speeding up inference compared to the default 32 bit weights.

Inputs are short audio clips, resampled to 16kHz internally. Outputs are the transcribed text, detected language, and processing duration.

A second, specialized path fine-tunes Whisper on Scottish accented English, targeting a documented

weakness of general purpose speech models: lower accuracy on accents underrepresented in training data (DiChristofano et al., 2022). Training data came from OpenSLR's UK and Ireland English Dialect dataset, 2,543 clips across male and female Scottish speakers.

Fine-tuning used LoRA, which freezes the base model and trains a small set of additional weights rather than the whole network, keeping the result small, a few megabytes. Training ran on a T4 GPU, separately from the CPU only deployment target: training happens where compute is fast and cheap, and the result is then run on the hardware the coursework constrains.

Word Error Rate was measured on a held out test set before and after training, using the same method both times. The base model scored 9.99% WER; fine-tuning dropped this to 5.66%, a 43% relative reduction. Some errors remain, for example one clip still confused "band" and "brand," evidence that fine-tuning reduced errors without removing all of them.

This accuracy has a real cost: the fine-tuned path uses the transformers library rather than faster-whisper, since LoRA adapters are not compatible with CTranslate2. On the same clip, the general endpoint returned a result in 0.51 seconds against 8.27 seconds for the fine-tuned endpoint, over sixteen times slower, a deliberate trade of accuracy for response time.

4 Development Process

Development followed the GitFlow branching model (Driessen, 2010). The main branch holds only production ready code, receiving merges only from a release branch at the end of the project; feature branches never merge into it directly. Ordinary development happens on develop, with a new feature branch for each unit of work, for example `feature/transcription-service` or `feature/model-finetuning`, merged back through a pull request once tested.

This was not followed perfectly: early on, a pull request was accidentally merged into main instead of develop, caught via the commit graph and fixed by resetting main to one commit. A few small fixes later also landed directly on develop rather than through their own branch. Later branches consistently followed the intended pattern.

5 Costs

The system's only resource-intensive service is transcription-service, which needs the full 4 vCPU, 16GB tier confirmed in Section 2. Pricing for that specification, an AWS m6i.xlarge in London, was confirmed using the AWS Pricing Calculator at $162.06 a month (Amazon Web Services, 2026). The lighter services, api-service and Postgres, Prometheus and Grafana combined, used under 250MB of memory and under 5% CPU during testing, so a smaller m6i.large instance covers all of them, confirmed at $81.03 a month each. Figure 2 shows the estimate.

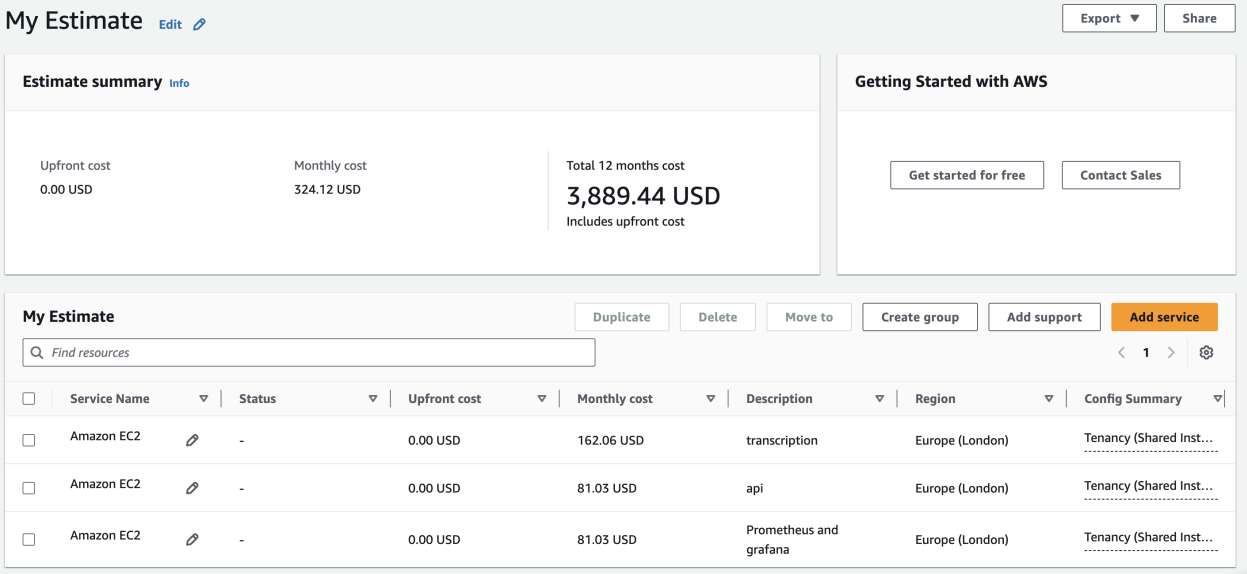

Figure 2: AWS Pricing Calculator estimate for the three instance types, Europe (London), on-demand, 100% utilization.

Following the module's pricing methodology (Menéndez, 2025a), infrastructure is sized to peak concurrent load while cost is spread across the full subscriber base, since not every subscriber transcribes at once. Each transcription-service instance handles 8 concurrent requests within its resource limit (Section 2). Assuming roughly 1% of subscribers have a request in flight at any time, the number of instances needed is:

$$N = \lceil R/8 \rceil, \qquad \text{Total} = 81.03 + 81.03 + (N \times 162.06)$$

where R is peak concurrent requests. Table 1 shows this applied at three subscriber counts.

Subscribers	Peak concurrent	Instances	Monthly cost	Per user
100	1	1	$324.12	$3.24
1,000	10	2	$486.18	$0.49
10,000	100	13	$2,268.84	$0.23

Table 1: Estimated monthly cost at three subscriber scales, using confirmed AWS Pricing Calculator figures.

Cost per subscriber falls as scale increases, since the fixed cost of the supporting services is spread across more users, the same pattern as the module's example.

6 Testing, Security, and Monitoring

Security is enforced in layers. Passwords are hashed with bcrypt, and JWT tokens authenticate every protected request. Rate limiting, using slowapi keyed by client IP, is set per endpoint by risk: 5 requests a minute on registration and 10 on login, where the risk is automated abuse, and 20 on transcription, mainly to protect against runaway cost on the most expensive endpoint. Upload validation checks file size (25MB cap) and declared content type before forwarding to the transcription service, rejecting bad input early rather than spending compute on it. This trusts the client's declared type rather than inspecting the file itself, a limitation discussed in Section 7.

Testing uses a pytest suite of nine tests against a temporary SQLite database and a mocked transcription service, checking the application's own logic without depending on other services being up. The most important test confirms one user cannot see another user's jobs, a genuine authorization check, not just a login requirement.

Monitoring uses Prometheus and Grafana. Both services expose `/metrics` through prometheus-fastapi-instrumentator (Prometheus Authors, 2024), needing only two lines of code per service. Prometheus scrapes both every 15 seconds, and Grafana (Grafana Labs, 2024) reads from it to show request rate, error rate, and 95th percentile latency. Two details matter: queries exclude Prometheus's own scrape traffic so the dashboard reflects real activity, and the error rate query returns zero rather than a blank panel when there are no errors, since Prometheus shows no data at all for an empty result, easily mistaken for a broken dashboard.

7 Limitations

Several parts of the system are not production ready. The JWT signing secret and the Grafana admin password both fall back to fixed defaults if no environment variable is set, fine for local testing but not for a real deployment.

Upload validation checks the file type a client declares, not the file's actual contents, so a client could mislabel a file and pass the check. A stronger version would inspect the file's header bytes rather than trust the label.

The system also does not scale automatically. The cost model in Section 5 assumes extra transcription-service instances as subscriber numbers grow, but nothing in the deployment does this on its own; a real autoscaling setup with a load balancer would be needed.

Postgres runs as a single instance with no replication, so a failure there would mean downtime and possible data loss.

8 Sustainability

Software Carbon Intensity, SCI = ((E $\times$ I) + M) / R, gives emissions per unit of use rather than a raw total (Green Software Foundation, 2023), where R is one transcription request.

M, the embodied emissions from hardware manufacturing, cannot be measured directly since AWS does not publish host specifications. This reuses the assumptions taught in the module's lecture (Menéndez, 2025b): a 4 vCPU allocation from a host with 32 total vCPUs and a 48 month lifespan, giving M = 3,204 gCO2e a month.

E is based on measured request times, 0.51 seconds general and 8.27 seconds Scottish, at an assumed 100W draw. At 1,000 subscribers and 20 requests a user a month, E = 0.714 kWh a month. Multiplied by the UK grid's carbon intensity, 122 gCO2e/kWh (National Energy System Operator, 2026), energy emissions come to 87.2 gCO2e a month.

SCI = (87.2 + 3204) / 20,000 = 0.165 gCO2e per request. Embodied emissions dominate by roughly 37 times, since the UK grid is comparatively clean, meaning the more effective lever is keeping hardware in use longer, not cutting energy use. At this scale, offsetting the monthly emissions would need roughly 1.43 mature trees.

References

Amazon Web Services (2026) *AWS Pricing Calculator*. Available at: `https://calculator.aws/` (Accessed: 26 July 2026).

DiChristofano, A., Shuster, H., Chandra, S. and Patwari, N. (2022) *Global Performance Disparities Between English-Language Accents in Automatic Speech Recognition*. arXiv:2208.01157. Available at: https://arxiv.org/abs/2208.01157 (Accessed: 26 July 2026).

Docker Inc. (2024) *Docker Documentation*. Available at: https://docs.docker.com/ (Accessed: 26 July 2026).

Driessen, V. (2010) *A Successful Git Branching Model*. Available at: https://nvie.com/posts/a-successful-git-branching-model/ (Accessed: 26 July 2026).

Grafana Labs (2024) *Grafana Documentation*. Available at: https://grafana.com/docs/ (Accessed: 26 July 2026).

Green Software Foundation (2023) *Software Carbon Intensity (SCI) Specification*. Available at: https://sci.greensoftware.foundation/ (Accessed: 26 July 2026).

Menéndez, H. (2025a) *Pricing your SaaS* [Lecture slides]. 5CCSACCA Cloud Computing for Artificial Intelligence, King's College London.

Menéndez, H. (2025b) *Sustainability* [Lecture slides]. 5CCSACCA Cloud Computing for Artificial Intelligence, King's College London.

National Energy System Operator (2026) *Great Britain's Monthly Electricity Stats*. Available at: https://www.neso.energy/energy-101/great-britains-monthly-energy-stats (Accessed: 26 July 2026).

OpenNMT (2024) *CTranslate2: Fast Inference Engine for Transformer Models*. Available at: https://github.com/OpenNMT/CTranslate2 (Accessed: 26 July 2026).

PostgreSQL Global Development Group (2024) *PostgreSQL 16 Documentation*. Available at: https://www.postgresql.org/docs/16/ (Accessed: 26 July 2026).

Prometheus Authors (2024) *Prometheus Documentation*. Available at: https://prometheus.io/docs/ (Accessed: 26 July 2026).

Radford, A., Kim, J.W., Xu, T., Brockman, G., McLeavey, C. and Sutskever, I. (2022) *Robust Speech Recognition via Large-Scale Weak Supervision*. arXiv:2212.04356. Available at: https://arxiv.org/abs/2212.04356 (Accessed: 26 July 2026).

Ramírez, S. (2024) *FastAPI Documentation*. Available at: https://fastapi.tiangolo.com/ (Accessed: 26 July 2026).